

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART VIII

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Collaborative Workflow: Claude Code + GitHub + Jira

Part – VIII: **Built-in Skills & Vuln-Web-App Current Status**



Vulnerable Web Application v2.0.0

Instructor: Muhammad Arif Butt, PhD

Vulnerable Web App – Version Roadmap

1 SECURITY HARDENING

Fix all 8 OWASP vulnerabilities, one tag at a time

v0.1.0 ★RELEASE	v0.1.1 ★RELEASE	v0.1.2	v0.1.3	v0.1.4	v0.1.5	v0.1.6	v0.1.7	v1.0.0 ★RELEASE
Baseline — 8 vulns open	**Dark Mode + bcrypt**	**Fix SQL Injection**	**Fix Exposed DB**	**Fix Session Hijacking**	**Fix Stored XSS**	**Fix Reflected XSS**	**Fix Rate Limiting**	**Fix CSRF — all 8 closed**
All 8 OWASP flaws present	MD5 → bcrypt (cost 12)	Parameterized queries	Removed /download/db	Env SECRET_KEY + random	HTML-escape username	HTML-escape /search	Per-IP limit, HTTP 429	Synchronizer token, 403

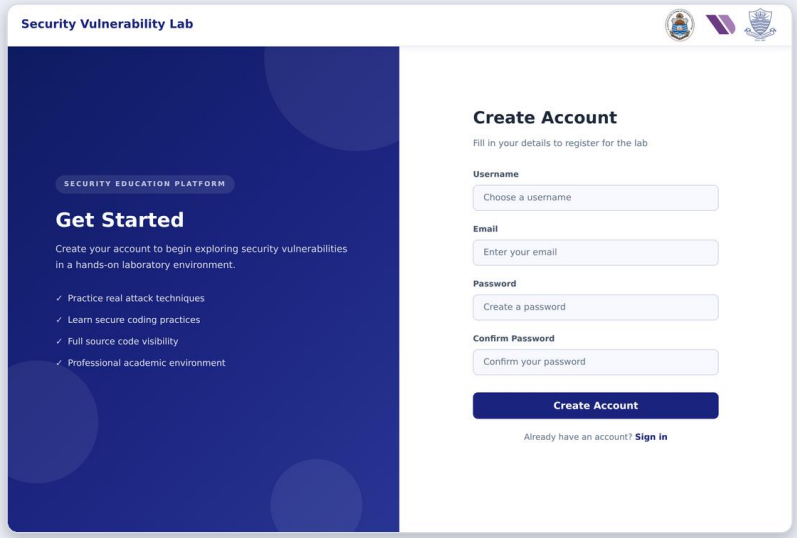
2 FEATURE BUILD-OUT

Add real authentication & account features

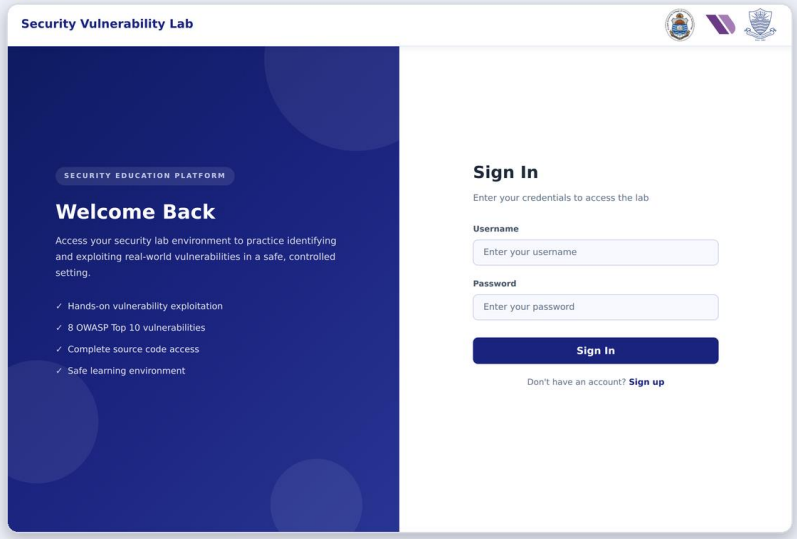
v1.0.1	v1.0.2	v1.0.3	v1.0.4	v1.0.5	v1.0.6	v1.0.7	v1.0.8	v2.0.0 ★RELEASE
Password Strength Meter	**User Profile Page**	**Google OAuth Login**	**Email Verification**	**Account Lockout**	**Email OTP 2FA**	**Authenticator App (TOTP)**	**QR Code Login**	**CAPTCHA on Login**
Advisory meter on signup	/profile + change password	OAuth 2.0 / OpenID Connect	Single-use 1-hour link	Lock after N failed logins	6-digit emailed code	TOTP + QR enrollment	Scan from trusted device	reCAPTCHA / hCaptcha

Vuln-Web-App v0.1.0

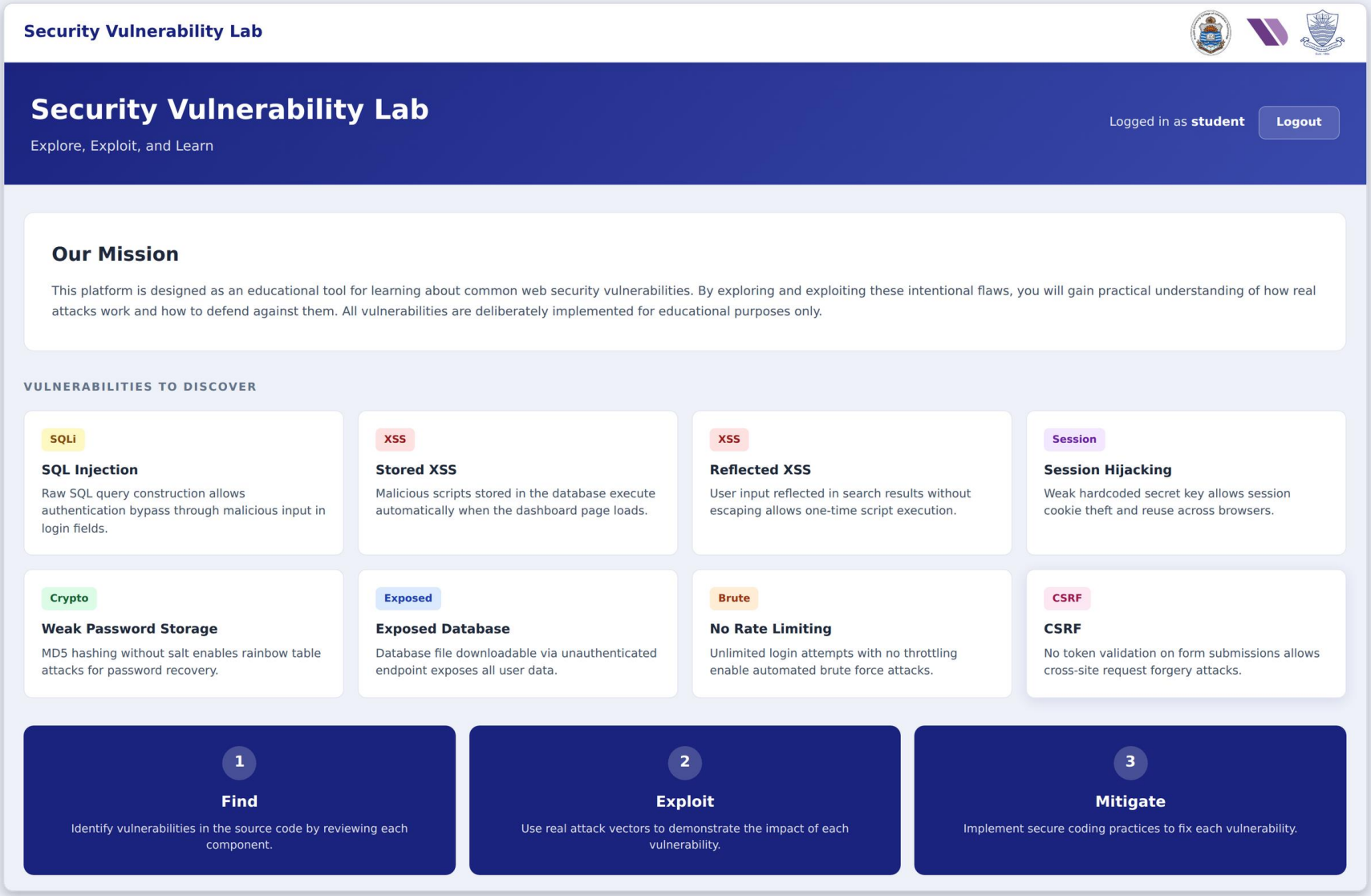
1 Sign Up /signup



2 Sign In /login

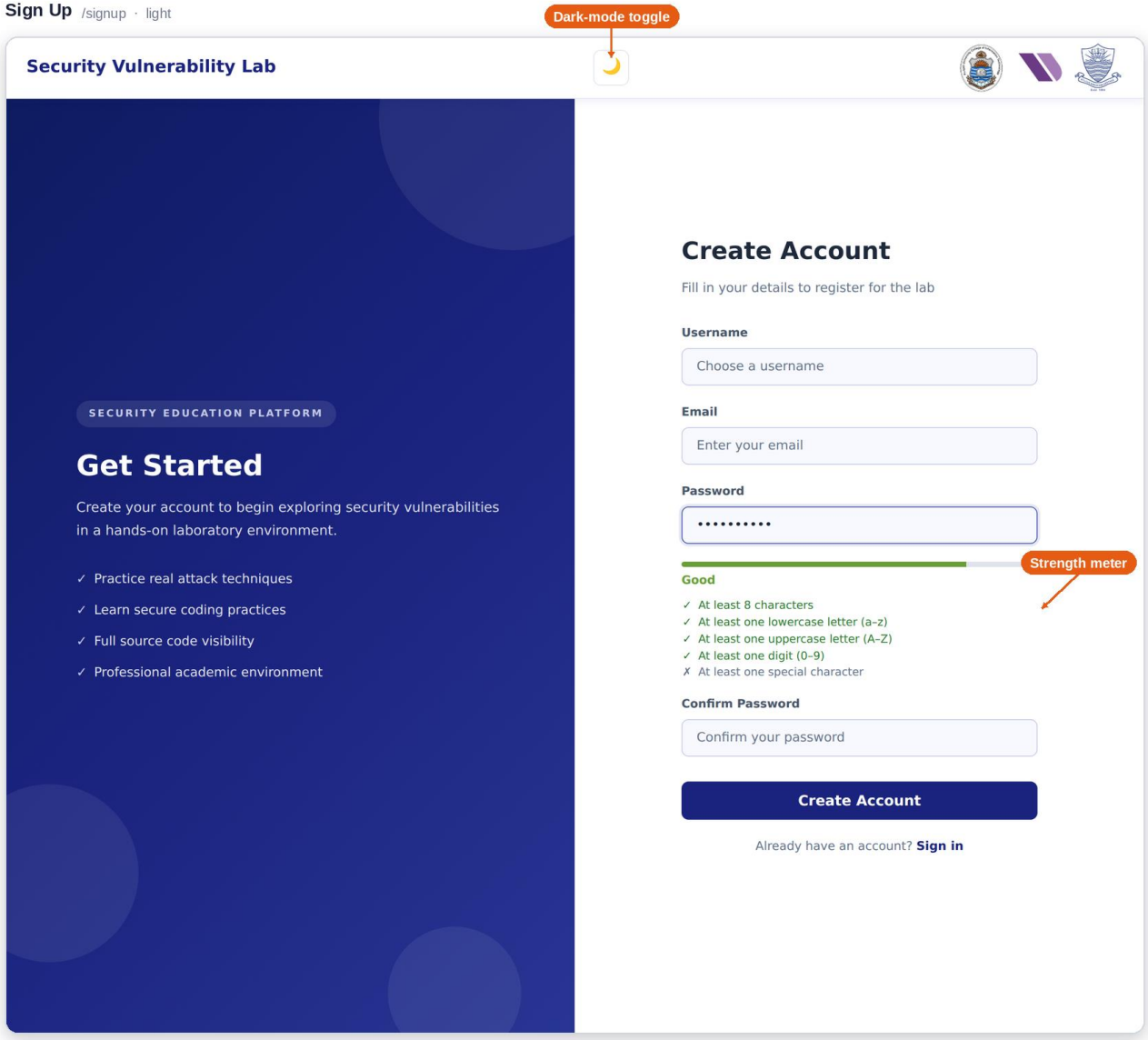


3 Landing Page — Dashboard /welcome

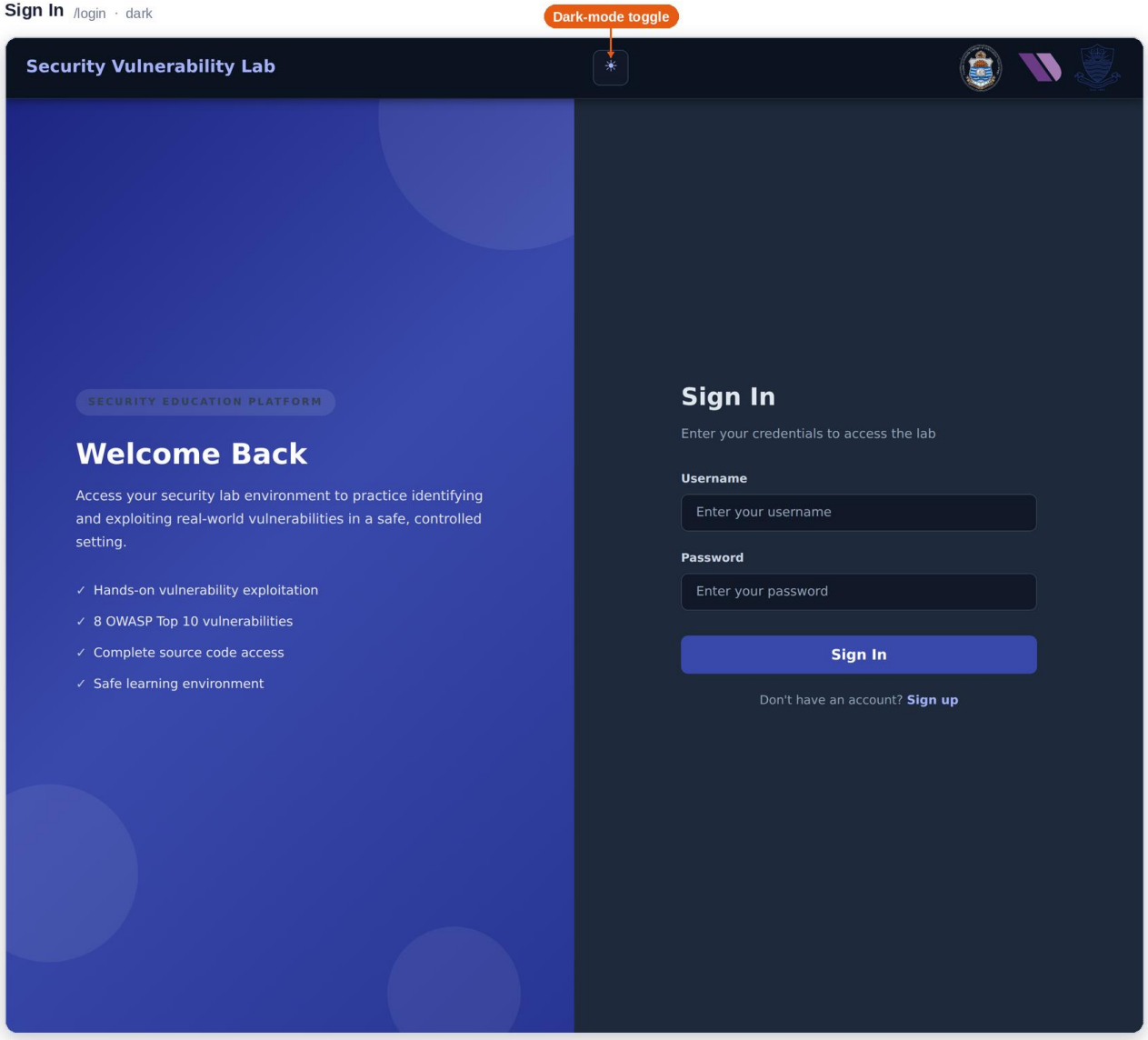


Vuln-Web-App v1.0.1

Sign Up /signup · light

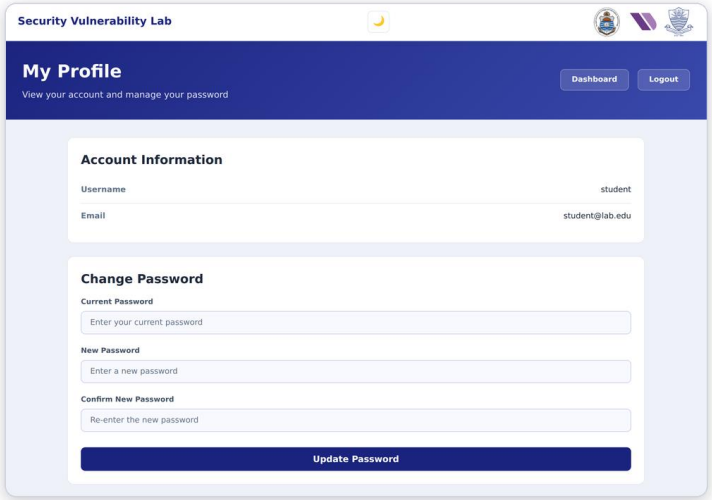


Sign In /login · dark

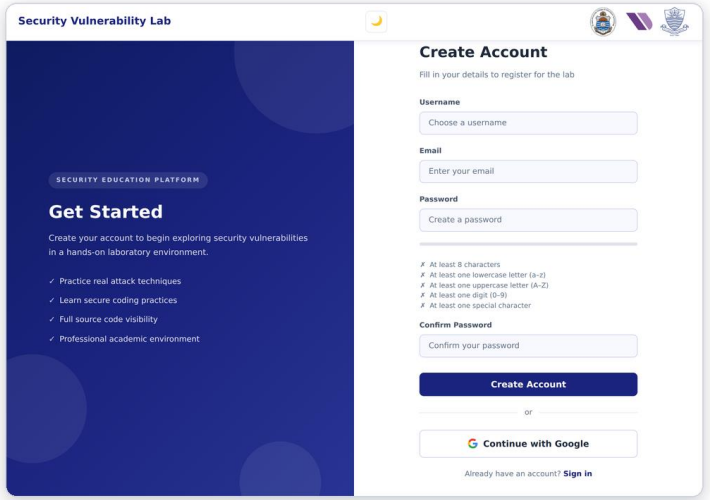


Vuln-Web-App v1.0.7

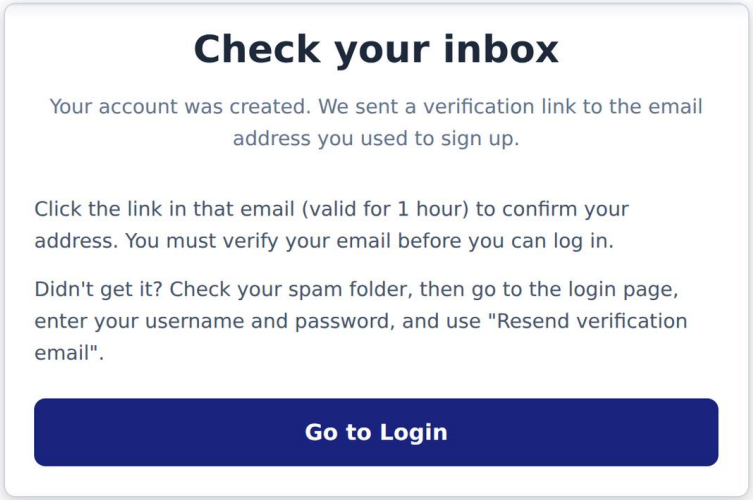
v1.0.2 User Profile Page



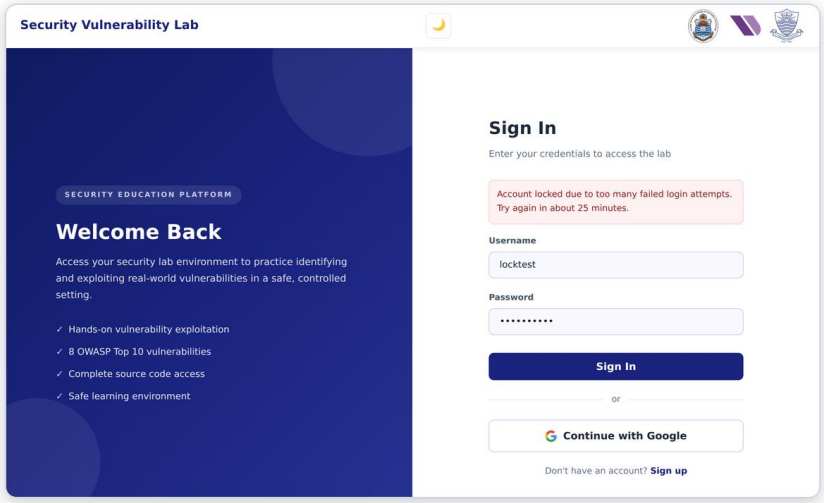
v1.0.3 Continue with Google · Sign Up



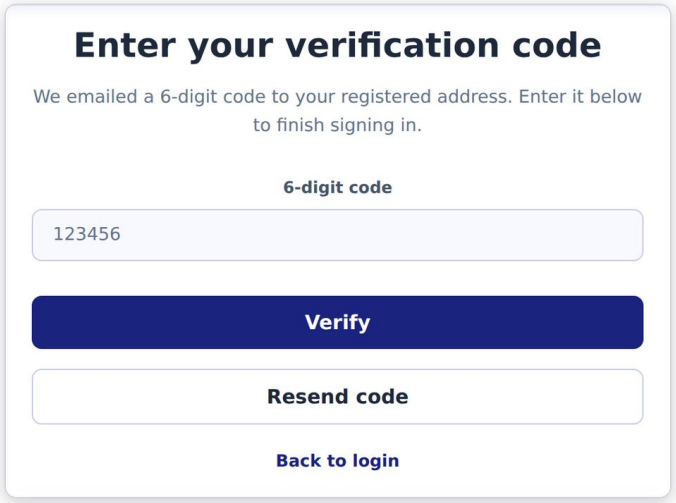
v1.0.4 Email Verification



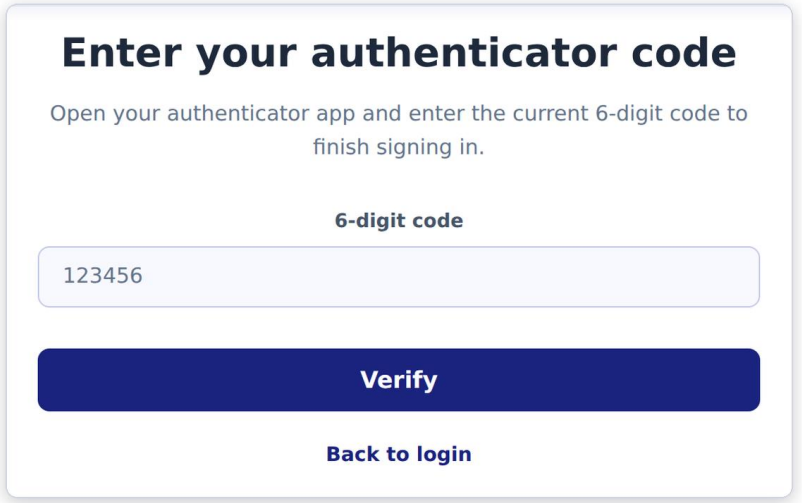
v1.0.5 Account Lockout



v1.0.6 Email OTP 2FA



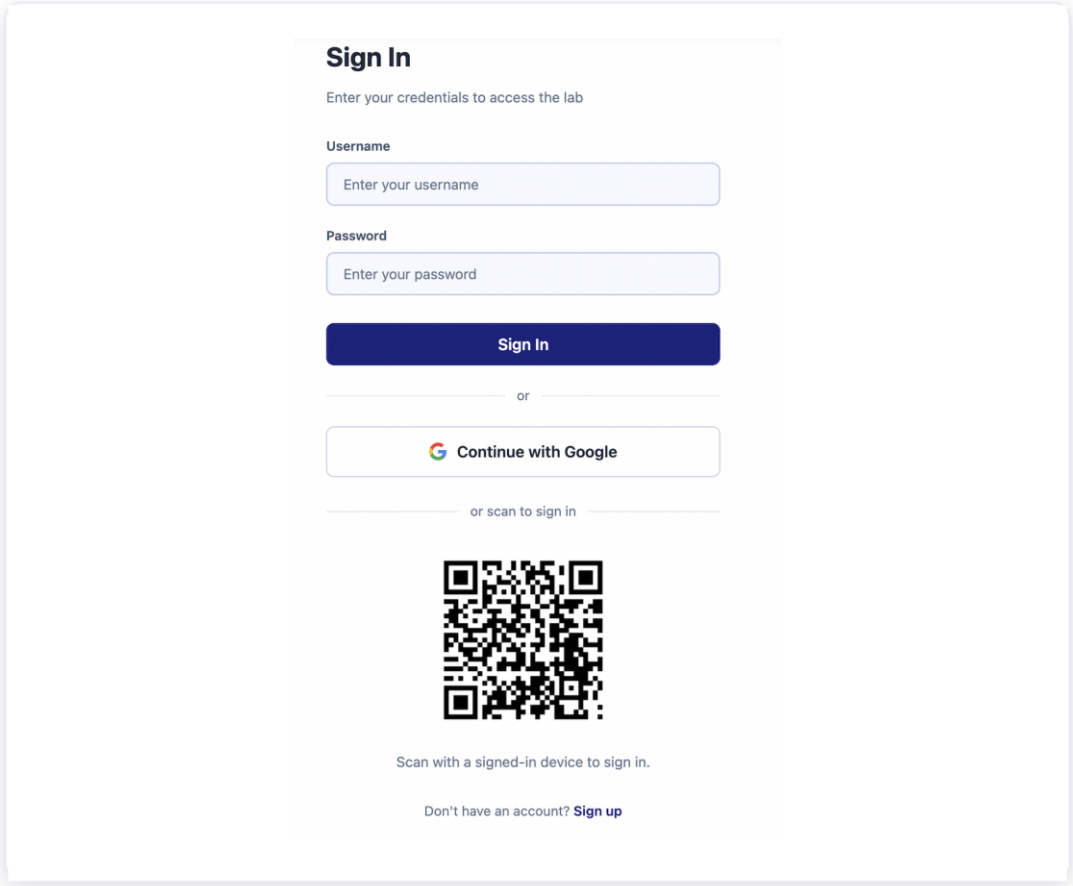
v1.0.7 Authenticator App · TOTP



Vuln-Web-App v2.0.0

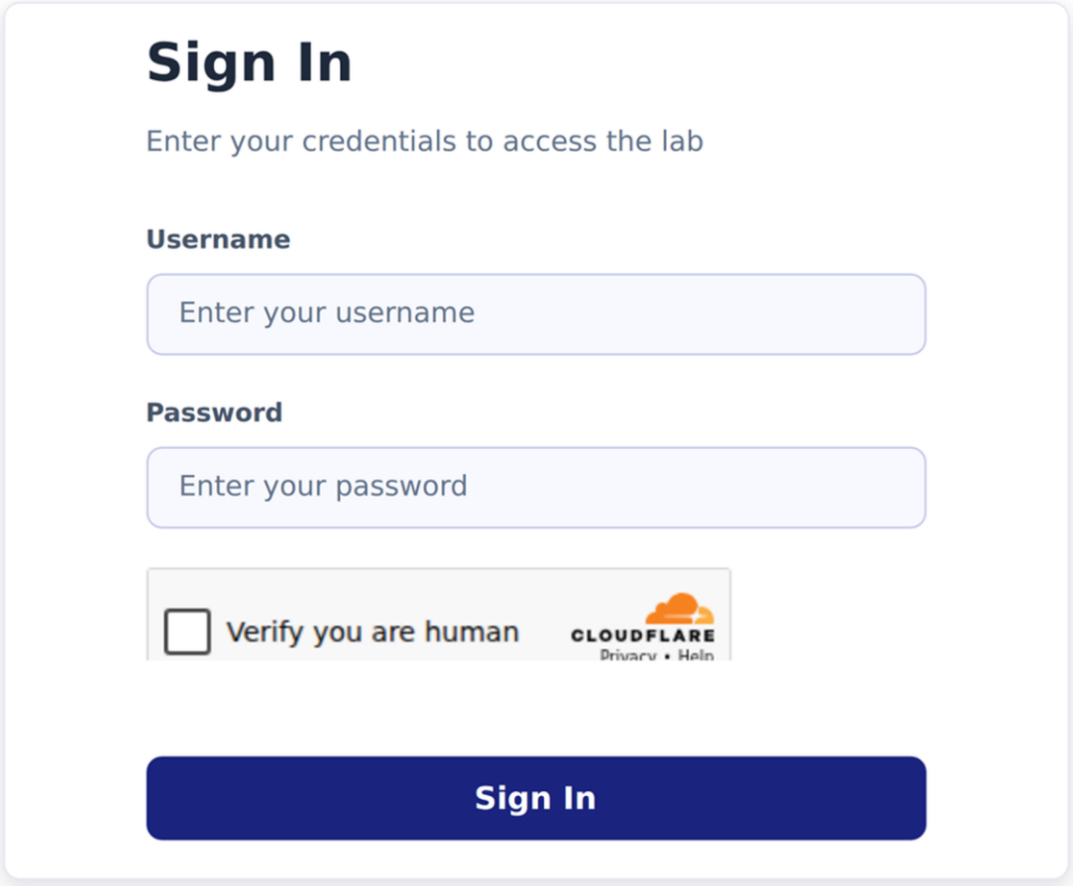
v1.0.8 QR-Code Login

Scan from an already signed-in device — passwordless, nothing typed on this browser.



v2.0.0 CAPTCHA on Login

Cloudflare Turnstile bot check, verified server-side before the password is ever checked.



https://github.com/arifpucit/vuln-web-app

https://arifpucit.github.io/auth-features/

https://arif-web-app.onrender.com/signup

Recap: Built-in Slash Commands

Recap: Manual Prompts to Slash Commands

Slash Commands are shortcuts you type inside Claude session, starting with /, that triggers a specific predefined action or workflow, without writing a full prompt. (user-invoked)

THE PROBLEM · Manual Workflows

You retype the same long, detailed prompt for every task.

"Explain this function…"
break down the inputs, outputs and logic, with clear examples

"Write unit tests…"
use pytest, cover edge cases, and report coverage

"Refactor this code…"
improve readability, remove redundancy, optimise

… then type it all again for the next task.

THE COST

- Slow
- Error-prone
- Repetitive
- Inconsistent

Slow, redundant, and easy to get wrong.

THE SHORTCUT

/tests

expands into

FULL PREDEFINED PROMPT

Write thorough unit tests with pytest, cover edge cases, and report coverage.

Claude runs it

Workflow runs

One short word stands in for a whole prompt — and the work runs automatically.

THE SOLUTION · Slash Commands

One command triggers the entire predefined workflow.

/explain	**Structured explanation** inputs, outputs & logic
/tests	**Full test suite** edge cases + coverage
/refactor	**Cleaner code** readable & efficient
/debug	**Bug found & fixed** trace, patch, verify

THE PAYOFF

- Fast
- Consistent
- Efficient
- Productive

Faster and repeatable — saves time, boosts productivity.

Recap: Built-in Slash Commands

Account & Configuration

/login
Sign in to your account

/logout
Sign out of your account

/config
Open Settings & preferences

/theme
Change the color theme

/model
Switch model, set default

/effort
Set the reasoning effort

/permissions
Allow / ask / deny rules

Extend & Connect

/mcp
Connect & manage MCP servers

/agents
Manage subagents

/hooks
Manage event-driven hooks

/plugin
Install & manage plugins

/add-dir
Add a working directory

Status & Diagnostics

/help
Usage help & command list

/status
Version, model & connectivity

/doctor
Diagnose the installation

/usage
Cost, limits & activity

/skills
List available skills

Session & History

/rename
Rename the current session

/clear
Start a fresh conversation

/resume
Reopen an earlier chat

/rewind
Roll back to a checkpoint

/export
Export conversation as text

Context & Memory

/context
Visualize the context window

/compact
Summarize to free context

/recap
One-line session summary

/btw
Side question, off the record

/init
Create the project CLAUDE.md

/memory
Edit CLAUDE.md memory files

Built-in/Bundled Skills

Building Blocks of Claude: Context Stack & Capability Extension

Skills are sets of instructions, but they also bundle supporting resources such as scripts and assets. Unlike slash commands, skills can be loaded automatically whenever a task or prompt matches the skill's description

Context Stack

What lives in the context window — and how it's managed

CLAUDE.md

Always loaded · passive

Loaded at the start of every session. Defines **project-wide context and conventions**. Loads hierarchically — user, project, and subdirectory files merge, with more specific files winning.

PURPOSE Persistent project knowledge and guidance.

Everything returns here. MCP tool definitions, tool results, and subagent summaries all flow back into this same window — which is exactly why progressive disclosure and isolation matter.

◄ BRIDGE — LIVES IN BOTH ►

Skills & Subagents count in each lens

Skills

User OR Claude invoked

Reusable procedures that **load only when needed** (progressive disclosure). Commands are now part of Skills — the manual `/invoke` entry point.

◄ CONTEXT STACK
Loads on demand; near-zero footprint until triggered.

CAPABILITY EXTENSION ►
Packages new, reusable workflows for Claude.

Subagents

Claude delegated

Run in their **own isolated context window** and return only a summary — the main thread stays clean.

◄ CONTEXT STACK
Isolates work; protects the main window.

CAPABILITY EXTENSION ►
Runs specialized & parallel tasks.

Capability Extension

What Claude can do, automate & ship

MCP Servers

Model invoked

Brokered access to external systems, APIs, databases, and developer tools.

PURPOSE External integrations via tool servers.

Hooks

Event triggered · deterministic

Deterministic code that fires on lifecycle events — formatting, logging, validation, and notifications. Enforce rules with code, not prompts.

PURPOSE Automate behavior at key workflow points.

Plugins — the distribution wrapper

Bundle **CLAUDE.md, skills, subagents, hooks, and MCP servers** into one installable, versioned unit — the canonical way to share a setup across teams and marketplaces. A container for all of the above, not a context primitive itself.

WRAPS ALL SIX

Right-sized context
Load only what's needed, when it's needed.

Better performance
Reduced tokens, faster responses.

Safer isolation
Subagents run in isolated context with clear boundaries.

Team leverage
Reuse expert workflows across projects and teams.

Composable
Skills can call other skills and tools.

Why use Skills?

A Skill is a reusable, file-based bundle of workflows, context, and best practices that Claude loads automatically; only when the task at hand actually calls for it, not kept in context all the time.

Why Skills?

Generalists, not specialists

LLMs reason and write well, but stumble on specialized, format-specific work like reports, slides, quizzes, and reviews.

One prompt, one time

A long, carefully crafted prompt gets the job done once — then disappears with the chat.

The work repeats

These tasks come back every day or week. Re-crafting the perfect prompt each time is slow and inconsistent.

Skills package the expertise

A reusable, file-based bundle of workflows, context, and best practices that Claude loads on demand.

Skills worth building

/quiz-generator

Generate MCQs and programming quizzes from any course topic.

/youtube-digest

Pull a YouTube transcript and summarize the key points with timestamps.

/pptx-deck

Turn rough notes or a document into a styled PowerPoint presentation.

/docx-report

Produce professional Word reports with headings, a table of contents, and tables.

/data-cleaner

Clean messy CSV/Excel data: fix types, deduplicate, normalize, and flag errors.

/diagram-it

Generate architecture, flow, and ER diagrams from code or a description.

Built-in Skills

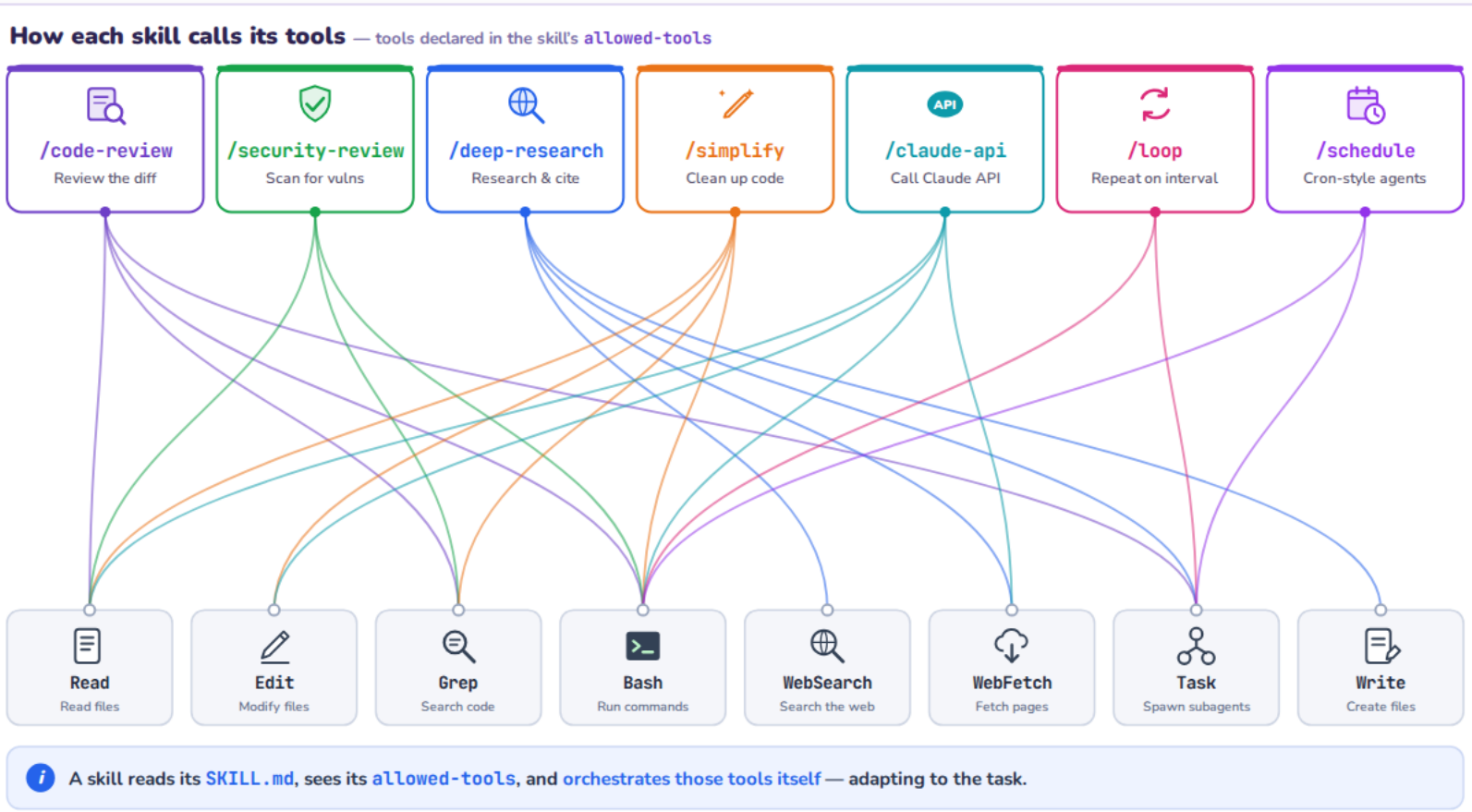
Skill	What it does	How it helps (example)	Outcome
/code-review	Reviews the current diff for correctness bugs and cleanups at a chosen effort level (low → ultra).	Pick an effort level — ultra fans out to a multi-agent cloud review low → med → high → ultra multi-agent cloud ⚠ bug · null deref · line 24 ✦ cleanup · dead var · line 51	Bugs caught before merge
/security-review	Runs a security review of the pending changes on the current branch.	Security findings on pending changes SQL injection — unsanitized input · auth.js:87 Hardcoded secret in config · config.ts:12	Vulnerabilities flagged early
/deep-research	Fans out web searches, fetches & fact-checks sources, then synthesizes a cited research report.	Research pipeline Search (fan-out) → Fetch → Fact-check → Synthesize → Cited report	Cited research report
/loop	Runs a prompt or slash command repeatedly on an interval — or self-paced if no interval is given.	Repeat on an interval (self-paced if none given) Run → wait 30m → Run → wait 30m → ...	Repeats until goal met
/simplify	Reuse, simplify, efficiency & altitude cleanups on changed code — **quality only, no bug hunting.**	Before → After quality only Before: `function f(a,b,c){ if(a){for(let i=0;i<b;i++){ if(c[i]>0){...}}}}` → After: `function f(a, b, c) { return c.filter(x => x > 0) .slice(0, b); }`	Cleaner, leaner code
/schedule	Create, update, list, or run scheduled cloud agents (cron-style routines), including one-time runs.	Cron-style scheduled cloud agents 0 9 * * 1 → every Mon 09:00 Weekly report Nightly tests One-time · Jul 1	Routines run on schedule
/claude-api	Helps write Anthropic API integrations.	Generate a streaming chat example with @anthropic-ai/sdk `import Anthropic from '@anthropic-ai/sdk'; const stream = await client.messages.create({ model: 'claude-opus-4', stream: true });`	Working API integration

How it works

1. You invoke a skill (like a slash command) e.g. `/code-review` → 2. Claude loads the skill's markdown instructions into context → 3. Claude reasons, plans, and uses tools to complete the task → 4. You get high-quality results

https://support.claude.com/en/

Built-in Skills in Action



Built-in Command vs Skill

who decides which tools run?

Built-in Command

- Runs **fixed, predefined logic**
- Deterministic — same input, same output
- Tool actions are hard-wired by the harness

e.g. `/clear` · `/compact` · `/model`

VS

our 7 live here

Skill

- `SKILL.md` = instructions + **allowed-tools**
- Claude **reads it & orchestrates** the tools
- Flexible, adaptive, context-aware

e.g. `/code-review` · `/deep-research` · `/simplify`

What happens when you run a skill

1. You type `/code-review`
2. Claude loads its `SKILL.md` into context
3. It reads the **allowed-tools** & instructions
4. Claude **orchestrates those tools** to do the job

Tools live inside the skill's `SKILL.md`:

```
---
name: code-review
allowed-tools: Read, Grep, Bash
---
Review the current diff for…
```

↳ The colored lines on the left are each skill's **allowed-tools**.

Reusable — Share skills across projects and teams.

Composable — Combine skills to build powerful workflows.

Context-Aware — Claude adapts execution to your codebase.

Safe & Controlled — Tools are sandboxed and permissioned.

Scalable — Bundle, version, and evolve skills as a team.

Hands on Practice: Built-in Skills



Q/A



Thank you!